# JOBSHEET XI

**Mata Kuliah: Praktikum Pemrograman Berbasis Objek**

*SOLID Principle dan Collections*

---

Dosen Pengampu:

**Dian Hanifudin Subhi, S.Kom., M.Kom.**




**Disusun oleh:**

Ahmad Rafid Riqkullah

254107020078

TI – 2B

D-IV Teknik Informatika

Tanggal: 05 November 2026

**POLITEKNIK NEGERI MALANG**

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

**TAHUN 2026-2027**

# C. Langkah Kerja

## Langkah 1: Bank Beralih dari Array ke Map

## Code:

Bank.java M  Main.java M

Jobsheet11 > id > ac > polinema > Bank.java > Bank

```java
Pidddd, 8 minutes ago | author (Pidddd)
package id.ac.polinema;

import java.util.LinkedHashMap;
import java.util.Map;

Pidddd, 8 minutes ago | author (Pidddd)
public class Bank {
    // Collections (LinkedHashMap) & Generics (<String, Account>) menggantikan array biasa
    private Map<String, Account> accounts = new LinkedHashMap<>();

    public boolean addAccount(Account account) {
        if (accounts.containsKey(account.getAccountNumber())) {
            return false;
        }
        // Menyimpan pasangan key (nomor rekening) dan value (objek Account) ke Map
        accounts.put(account.getAccountNumber(), account);
        return true;
    }

    public Account findAccount(String accountNumber) {
        // Pencarian langsung via key tanpa perlu looping array
        return accounts.get(accountNumber);
    }

    public void printAllAccounts() {
        // Iterasi nilai di dalam Map menggunakan values()
        for (Account acc : accounts.values()) {
            acc.printInfo();
        }
    }

    public void printMonthlyFees() {
        for (Account acc : accounts.values()) {
            System.out.println(acc.getAccountNumber() + " fee: " + acc.monthlyFee());
        }
    }

    public void processMonthEnd() {
        for (Account acc : accounts.values()) {
            if (acc instanceof InterestBearing bearing) {
                bearing.applyInterest();
                System.out.println(acc.getAccountNumber() + " interest applied, new balance: " + acc.getBalance());
            }
            System.out.println(acc.getAccountNumber() + " monthly fee: " + acc.monthlyFee());
        }
    }
}
    Pidddd, 8 minutes ago • Jobsheet 11 Template
```

*Gambar 1 Penerapan LinkedHashMap pada Bank.java*

```java
        // Instansiasi Bank tanpa parameter kapasitas karena Map berukuran dinamis
        Bank bank = new Bank();
        bank.addAccount(acc1);
        bank.addAccount(acc2);
        bank.addAccount(savings);
        bank.processMonthEnd();
    }
}
```

*Gambar 2 Instansiasi objek Bank tanpa parameter kapasitas di Main.java*

## Hasil:

Problems 34  Output  Debug Console  Terminal  Ports  Comments

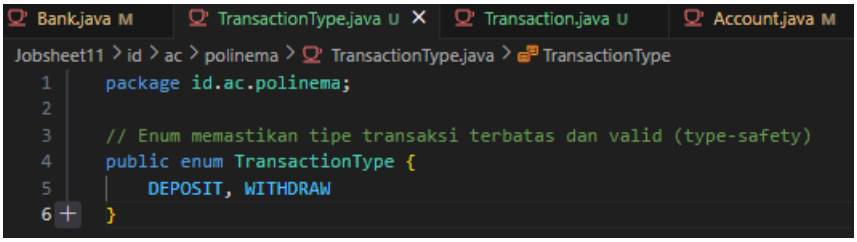
```
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-> &
ava-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\
torage\641953bbf92bcda82cdde5d7c75e879e\redhat.java\jdt_ws\Praktikum-Pemrograman-Berbasi-Object-PBO-_2f5b4ee8\bin' 'id.
Withdrawal failed: A003: insufficient balance for a withdrawal of 70000.0
Withdrawal succeeded, new balance: 70000.0
A001 interest applied, new balance: 505000.0
A001 monthly fee: 0.0
A002 monthly fee: 15000.0
A003 interest applied, new balance: 71400.0
A003 monthly fee: 0.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO->
```

*Gambar 3 Output terminal Langkah 1*

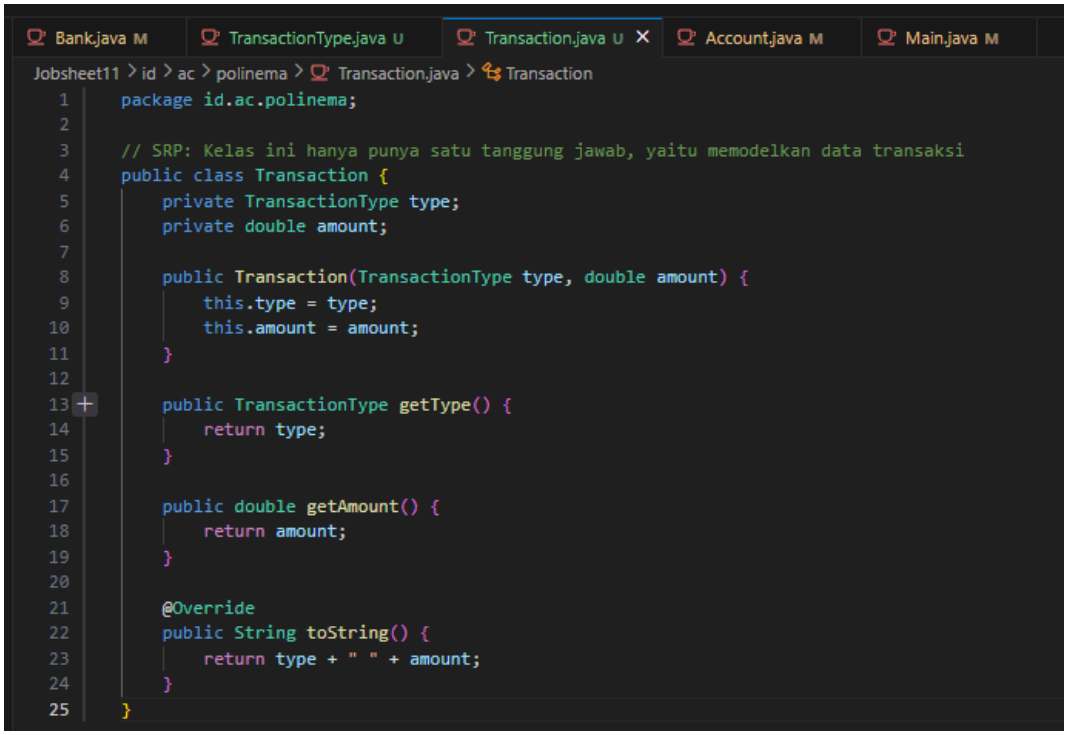
**Penjelasan**: Penyimpanan rekening pada kelas Bank diubah dari array berukuran tetap (Account[]) menjadi LinkedHashMap<String, Account>. Keuntungan perubahan ini adalah kapasitas penyimpanan menjadi dinamis tanpa perlu menentukan batas ukuran di awal. Selain itu, pencarian data rekening melalui nomor rekening (accountNumber sebagai *key*) menjadi langsung dan instan via accounts.get() tanpa perlu melakukan penelusuran satu per satu (*linear scan*). Implementasi LinkedHashMap juga memastikan urutan rekening tetap terjaga sesuai urutan saat didaftarkan (*insertion order*).

### Langkah 2: Transaction, Single Responsibility Principle

**Code**:




*Gambar 4 Enum TransactionType.java*




*Gambar 5 Kelas Transaction.java untuk data mutasi (SRP)*

```java
package id.ac.polinema;

import java.util.ArrayList;
import java.util.List;

public abstract class Account {
    private String accountNumber;
    private Customer owner;
    private double balance;

    // Collections: List & ArrayList menyimpan objek Transaction
    private List<Transaction> history = new ArrayList<>();

    public Account(String accountNumber, Customer owner, double balance) {
        this.accountNumber = accountNumber;
        this.owner = owner;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public Customer getOwner() {
        return owner;
    }

    public double getBalance() {
        return balance;
    }

    public List<Transaction> getHistory() {
        return history;
    }

    public boolean deposit(double amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        // SRP: Pencatatan mutasi transaksi dilakukan saat setoran berhasil
        history.add(new Transaction(TransactionType.DEPOSIT, amount));
        return true;
    }

    public boolean deposit(double amount, String note) {
        if (!deposit(amount)) {
            return false;
        }
        System.out.println(accountNumber + " deposit note: " + note);
        return true;
    }

    public void withdraw(double amount) throws InsufficientBalanceException {
        if (!canWithdraw(amount)) {
            throw new InsufficientBalanceException(
                    accountNumber + ": insufficient balance for a withdrawal of " + amount);
        }
        balance -= amount;
        // SRP: Pencatatan mutasi transaksi dilakukan saat penarikan berhasil
        history.add(new Transaction(TransactionType.WITHDRAW, amount));
    }

    protected boolean canWithdraw(double amount) {
        return amount > 0 && amount <= balance;
    }

    public void printInfo() {
        System.out.println(accountNumber + " - " + owner.getName() + " - balance: " + balance);
    }

    public abstract double monthlyFee();
}
```

*Gambar 6 Pencatatan riwayat transaksi pada Account.java*

```java
    }

    // Menampilkan riwayat transaksi dari objek Account
    public void printHistory(String accountNumber) {
        Account acc = findAccount(accountNumber);
        if (acc == null) {
            System.out.println("Account not found: " + accountNumber);
            return;
        }
        for (Transaction t : acc.getHistory()) {
            System.out.println(accountNumber + " " + t);
        }
    }
}       Pidddd, 21 minutes ago • Langkah 1: Bank Beralih dari Array ke Map
```

*Gambar 7 Method printHistory() pada Bank.java*

```java
        bank.processMonthEnd();

        // Langkah 2: Menampilkan riwayat mutasi transaksi rekening A003
        bank.printHistory("A003");
    }
}       Pidddd, 22 minutes ago • Langkah 1: Bank Beralih dari Array ke Map
```

*Gambar 8 Pemanggilan printHistory() di Main.java*

**Hasil**:

```
gravity IDE\User\workspaceStorage\641953bbf92bcda82cdde5d7c75e879e\redhat.java\jdt_ws\Praktikum-Pemrograman-Berbasi-Obj
Withdrawal failed: A003: insufficient balance for a withdrawal of 70000.0
Withdrawal succeeded, new balance: 70000.0
A001 interest applied, new balance: 505000.0
A001 monthly fee: 0.0
A002 monthly fee: 15000.0
A003 interest applied, new balance: 71400.0
A003 monthly fee: 0.0
A003 WITHDRAW 30000.0
A003 DEPOSIT 1400.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO->
3 needs reviewers  ⊗ 34 ⚠ 0  Java: Ready
```

*Gambar 9 Output terminal riwayat mutasi rekening A003*

**Penjelasan**: Penerapan *Single Responsibility Principle* (SRP) dilakukan dengan memisahkan representasi data transaksi ke kelas tersendiri, yaitu Transaction dan TransactionType (enum). Kelas Account tidak lagi memikul tanggung jawab atas format data transaksi, melainkan hanya menampung riwayatnya ke dalam List<Transaction>. Pencatatan mutasi transaksi ke dalam riwayat (history.add) hanya dieksekusi setelah validasi saldo atau penarikan berhasil, sehingga transaksi yang gagal tidak ikut tercatat.

### Langkah 3: AccountRepository, Dependency Inversion Principle

**Code**:

Bank.java M | TransactionType.java | Transaction.java | Account.java | Main.java M | AccountRepository.jav

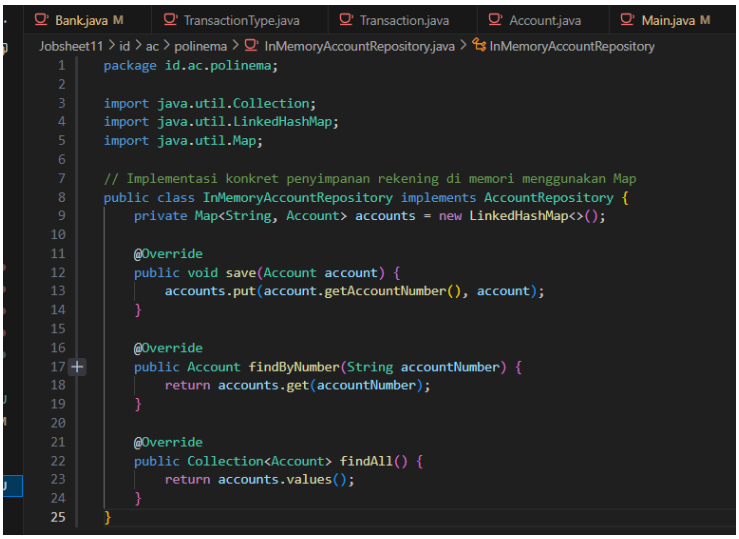
Jobsheet11 > id > ac > polinema > AccountRepository.java > AccountRepository

```java
package id.ac.polinema;

import java.util.Collection;

// DIP: Abstraksi penyimpanan data agar kelas tingkat tinggi tidak bergantung pada modul konkret
public interface AccountRepository {
    void save(Account account);
    Account findByNumber(String accountNumber);
    Collection<Account> findAll();
}
```

*Gambar 10 Interface AccountRepository.java (DIP)*




*Gambar 11 Kelas InMemoryAccountRepository.java*




*Gambar 12 Penggunaan interface repository pada Bank.java*

```
27          // Langkah 3: Mengirim implementasi konkret repository ke konstruktor Bank
28          Bank bank = new Bank(new InMemoryAccountRepository());
29
30          // Langkah 1: Penambahan akun dan pemrosesan akhir bulan
31          bank.addAccount(acc1);
32          bank.addAccount(acc2);
33          bank.addAccount(savings);      Pidddd, 10 minutes ago • Langkah 2: Transaction, 
34          bank.processMonthEnd();
35
36          // Langkah 2: Menampilkan riwayat transaksi A003
37          bank.printHistory("A003");
38      }
39  }
```

*Gambar 13 Injeksi InMemoryAccountRepository di Main.java*

**Hasil**:

```
Problems 34   Output   Debug Console   Terminal   Ports   Comments

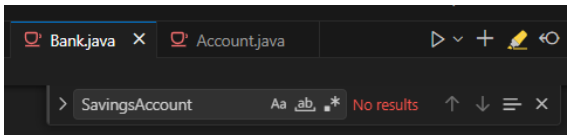
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-> &
ava-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\I
torage\641953bbf92bcda82cdde5d7c75e879e\redhat.java\jdt_ws\Praktikum-Pemrograman-Berbasi-Object-PBO-_2f5b4ee8\bin' 'id.a
Withdrawal failed: A003: insufficient balance for a withdrawal of 70000.0
Withdrawal succeeded, new balance: 70000.0
A001 interest applied, new balance: 505000.0
A001 monthly fee: 0.0
A002 monthly fee: 15000.0
A003 interest applied, new balance: 71400.0
A003 monthly fee: 0.0
A003 WITHDRAW 30000.0
A003 DEPOSIT 1400.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO->
```

*Gambar 14 Output terminal Langkah 3*

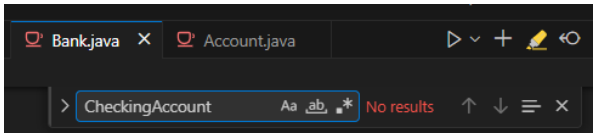
**Penjelasan**: Penerapan *Dependency Inversion Principle* (DIP) dilakukan dengan membuat kelas Bank (modul tingkat tinggi) bergantung pada abstraksi berupa interface AccountRepository, bukan langsung pada struktur data konkret seperti Map. Mekanisme penyimpanan nyata dikelola oleh kelas InMemoryAccountRepository yang disuntikkan (*dependency injection*) ke Bank melalui konstruktor. Dengan pemisahan ini, cara penyimpanan data dapat diganti (misalnya ke basis data) di kemudian hari tanpa perlu mengubah satu baris pun kode logika di dalam kelas Bank.

## Langkah 4: Menelusuri OCP, LSP, dan ISP pada Kode yang Sudah Dibangun

### 4a. Open / Closed



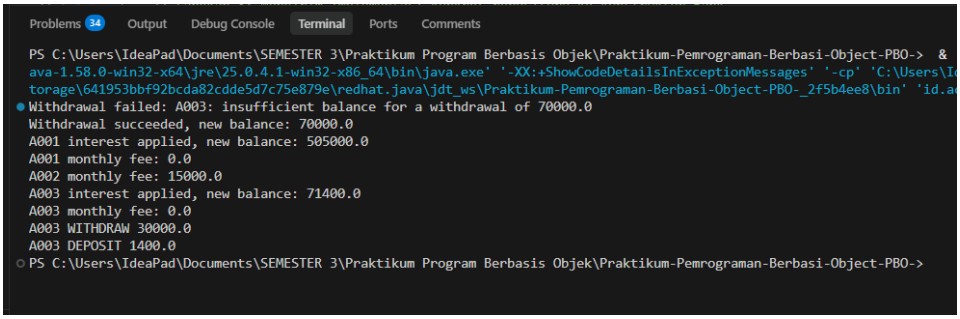

*Gambar 15 Pencarian SavingsAccount*




*Gambar 16 Pencarian CheckingAccount*

**Penjelasan**: Hasil pencarian kata kunci SavingsAccount dan CheckingAccount pada berkas Bank.java dan Account.java menunjukkan *No results*. Hal ini membuktikan berlakunya *Open/Closed Principle* (OCP), di mana sistem terbuka untuk diperluas dengan menambahkan subclass rekening baru beserta aturan penarikannya via *method overriding*, namun tertutup dari modifikasi kode lama pada kelas Bank dan Account.

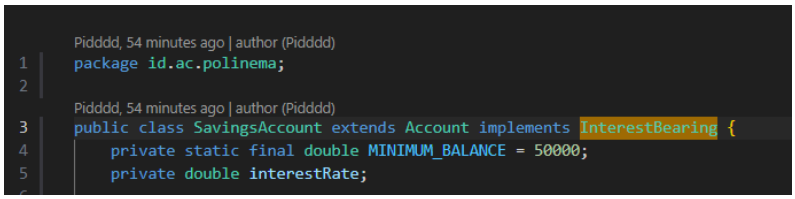
### 4b. Liskov Substitution



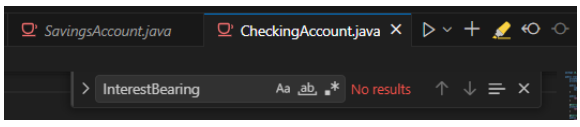

*Gambar 17 Output eksekusi polimorfik monthlyFee() (LSP)*

**Penjelasan**: Pada method processMonthEnd(), pemanggilan acc.monthlyFee() dilakukan secara seragam lewat tipe referensi superclass Account tanpa bantuan percabangan if-else untuk membedakan jenis akun. Baik SavingsAccount (biaya 0.0) maupun CheckingAccount (biaya 15000.0) dapat menggantikan posisi Account secara mulus dan mengeksekusi logikanya masing-masing dengan benar tanpa merusak program.

### 4c. Interface Segregation




*Gambar 18 Implementasi InterestBearing pada SavingsAccount.java*

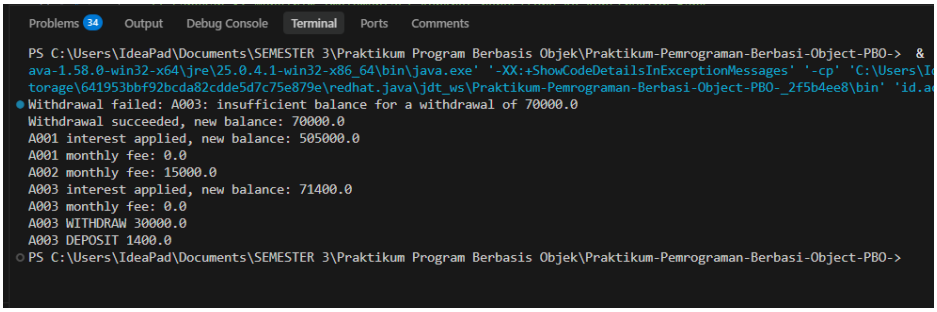



*Gambar 19 Pencarian InterestBearing di CheckingAccount.java*

**Penjelasan**: Interface InterestBearing dirancang kecil dan spesifik hanya dengan satu method applyInterest(). Interface ini hanya diimplementasikan oleh kelas yang relevan (SavingsAccount), sementara kelas CheckingAccount tidak dipaksa untuk mengimplementasikannya. Hal ini membuktikan *Interface Segregation Principle* (ISP), yaitu mencegah kelas dibebani oleh kontrak method yang tidak dibutuhkannya.

## D. Tugas dan Hasil Kerja

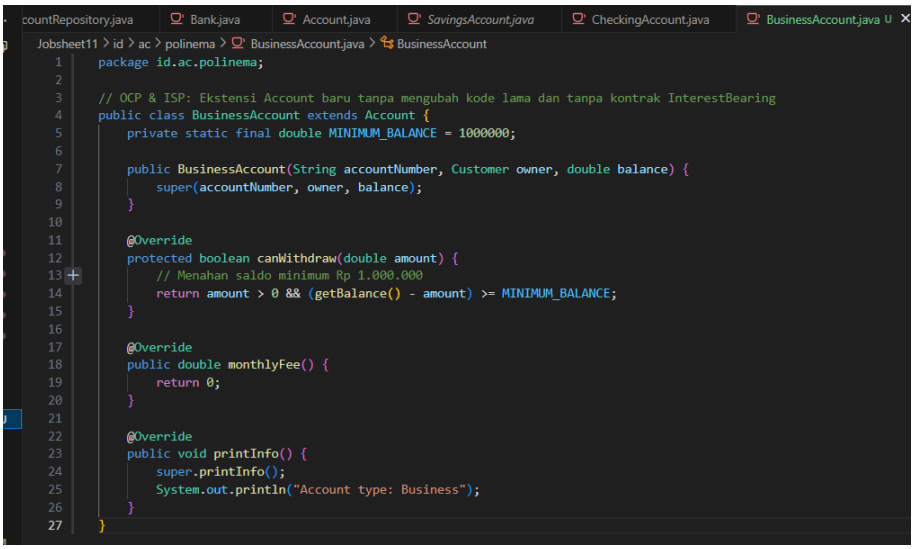
- Screenshot output program setelah Langkah 3.




*Gambar 20 Output program setelah Langkah 3*

- Tugas mandiri:

(a) BusinessAccount: membuktikan OCP, LSP, dan ISP lewat kode baru. Tambahkan BusinessAccount extends Account (rekening bisnis, saldo minimum Rp 1.000.000, tidak berbunga). Diagram berikut hanya sketsa method yang perlu di-override, BUKAN kode jadi, implementasi canWithdraw()/monthlyFee()/printInfo() (persis pola SavingsAccount / CheckingAccount yang sudah ditelusuri di Langkah 4) diserahkan sepenuhnya padamu:




*Gambar 21 Implementasi kelas BusinessAccount.java*

Daftarkan ke Bank, lalu jalankan processMonthEnd() dan printAllAccounts():

```
        }

        // Tugas Mandiri: Menambahkan objek BusinessAccount (A004)
        Customer customer4 = new Customer("Budi", "0812-0000-0004");
        BusinessAccount business = new BusinessAccount("A004", customer4, 2000000);

        // Langkah 3: Mengirim implementasi konkret repository ke konstruktor Bank
        Bank bank = new Bank(new InMemoryAccountRepository());

        // Langkah 1 & Tugas Mandiri: Penambahan akun dan pemrosesan akhir bulan
        bank.addAccount(acc1);
        bank.addAccount(acc2);
        bank.addAccount(savings);
        bank.addAccount(business);
        bank.processMonthEnd();
        bank.printAllAccounts();

        // Langkah 2: Menampilkan riwayat transaksi A003
        bank.printHistory("A003");
    }
}
```

*Gambar 22 Modifikasi Main.java untuk pendaftaran BusinessAccount*

Buktikan dua hal berikut dari output program, tanpa mengubah satu baris pun Account.java atau Bank.java:

- (Open/Closed + Liskov Substitution) baris A004 monthly fee: 0.0 muncul di antara baris processMonthEnd() yang lain, dan A004 - Budi - balance: 2000000.0 diikuti Account type: Business muncul di printAllAccounts(), persis seperti SavingsAccount/CheckingAccount diproses sebelumnya.
- (Interface Segregation) tidak ada baris A004 interest applied yang muncul, sebab BusinessAccount tidak meng-implement InterestBearing.

**Hasil:**

```
Withdrawal failed: A003: insufficient balance for a withdrawal of 70000.0
Withdrawal succeeded, new balance: 70000.0
A001 interest applied, new balance: 505000.0
A001 monthly fee: 0.0
A002 monthly fee: 15000.0
A003 interest applied, new balance: 71400.0
A003 monthly fee: 0.0
A004 monthly fee: 0.0
A001 - Nadia - balance: 505000.0
Account type: Savings, interest rate: 0.01
A002 - Sari - balance: 200000.0
Account type: Checking, overdraft limit: 50000.0
A003 - Rian - balance: 71400.0
Account type: Savings, interest rate: 0.02
A004 - Budi - balance: 2000000.0
Account type: Business
A003 WITHDRAW 30000.0
A003 DEPOSIT 1400.0
PS C:\Users\IdeaPad\Documents\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemro
```

*Gambar 23 Gambar 23 Output terminal pengujian BusinessAccount*

**Penjalasan**: Kelas BusinessAccount ditambahkan sebagai subclass baru dari Account dengan aturan saldo minimum Rp 1.000.000 tanpa mengubah kode pada Account.java maupun Bank.java. Hasil eksekusi membuktikan tiga prinsip sekaligus:

- **OCP & LSP:** Objek BusinessAccount (A004) dapat langsung ditambahkan dan diproses secara polimorfik di dalam processMonthEnd() (menampilkan A004 monthly fee: 0.0) serta printAllAccounts() (menampilkan informasi saldo dan Account type:

Business) menggantikan tipe referensi Account tanpa merusak alur program.

- **ISP:** Tidak ada baris A004 interest applied pada output karena BusinessAccount tidak mengimplementasikan interface InterestBearing.

(b) Jawab secara singkat (2-3 kalimat untuk masing-masing pertanyaan):

(a) sebutkan prinsip SOLID mana yang dibuktikan oleh fakta bahwa Account.java/Bank.java tidak berubah sama sekali setelah BusinessAccount ditambahkan, dan jelaskan mengapa.

**Jawab:** Fakta tersebut membuktikan Open/Closed Principle (OCP) dan Liskov Substitution Principle (LSP). Sistem terbuka untuk perluasan (*open for extension*) lewat penambahan subclass BusinessAccount, namun tertutup untuk modifikasi (*closed for modification*) karena Bank memproses seluruh rekening melalui abstraksi superclass Account yang dapat langsung digantikan oleh subclass mana pun secara valid.

(b) processMonthEnd() memakai instanceof InterestBearing, bukan instanceof SavingsAccount. Jelaskan mengapa BusinessAccount otomatis diperlakukan benar (tidak dikenai bunga) tanpa Bank.java perlu tahu apa pun tentang keberadaannya.

**Jawab**: Karena processMonthEnd() memeriksa kapabilitas objek berdasarkan kontrak interface (InterestBearing), bukan memeriksa nama kelas konkret tertentu. Berkat Interface Segregation Principle (ISP) dan OCP, BusinessAccount yang tidak mengimplementasikan InterestBearing akan otomatis menghasilkan nilai false pada pengecekan instanceof, sehingga dilewati dari perhitungan bunga tanpa Bank.java perlu mengenali kelas BusinessAccount.

(c) Suatu saat nanti, InMemoryAccountRepository akan diganti kelas lain yang menyimpan data ke database. Jelaskan mengapa Bank.java tidak perlu diubah satu baris pun untuk pergantian itu, dan prinsip SOLID mana yang membuat ini mungkin.

**Jawab**: Hal ini dimungkinkan oleh Dependency Inversion Principle (DIP). Kelas Bank hanya bergantung pada abstraksi berupa interface AccountRepository, bukan pada kelas implementasi konkretnya. Selama kelas penyimpanan database yang baru mengimplementasikan kontrak method pada AccountRepository, pergantian objek cukup dilakukan saat injeksi di Main.java tanpa mengubah satu baris kode pun di dalam Bank.java.

## Hasil dan Pembahasan

Praktikum ini menerapkan struktur data dari Java Collections Framework serta kelima prinsip desain berorientasi objek (SOLID Principles):

- Collections & Generics: Penggunaan LinkedHashMap<String, Account> dan ArrayList<Transaction> menggantikan array statis, sehingga kapasitas penyimpanan bersifat dinamis, pencarian data melalui *key* berjalan instan tanpa *linear scan*, urutan elemen terjaga, dan keamanan tipe data (*type-safety*) terjamin sejak tahap kompilasi.
- SOLID Principles: Pemisahan pencatatan mutasi ke kelas Transaction menerapkan SRP, sedangkan pemisahan penyimpanan ke interface AccountRepository menerapkan DIP. Selain itu, penambahan BusinessAccount membuktikan bahwa OCP, LSP, dan ISP bekerja secara sinergis sehingga fitur baru dapat diintegrasikan tanpa memodifikasi kode kelas utama (Account dan Bank).

## Kesimpulan

1. Penggunaan koleksi Map dan List dengan *Generics* meningkatkan efisiensi pencarian data serta fleksibilitas pengelolaan objek dibandingkan array konvensional berukuran tetap.
2. Penerapan kelima prinsip SOLID (**SRP, OCP, LSP, ISP, dan DIP**) menghasilkan arsitektur kode yang modular, rendah ketergantungan (*loosely coupled*), dan mudah dikembangkan (*scalable*) tanpa berisiko merusak fungsionalitas sistem yang sudah berjalan.

## Lampiran

Link Github Ahmad Rafid Riqkullah atau https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet11/id/ac/polinema